

50 CI/CD MISTAKES AND HOW TO FIX THEM

REAL PROBLEMS | REAL CAUSES | REAL SOLUTIONS
FOR SENIOR ENGINEERS




SECURITY
BEST PRACTICES


RELIABILITY
INSIGHTS


TROUBLESHOOTING
GUIDE


SCALING
CONSIDERATIONS


PRODUCTION
LESSONS

- ✓ AVOID COMMON PITFALLS
- ✓ BUILD MORE RELIABLE PIPELINES
- ✓ DEPLOY WITH CONFIDENCE

50
REAL MISTAKES
REAL SOLUTIONS

CI/CD FAILURE STARTS WITH PIPELINE DESIGN

3 COMMON MISTAKES THAT CREATE BIG PROBLEMS

1

TREATING THE PIPELINE AS A COLLECTION OF SCRIPTS INSTEAD OF A PRODUCTION SYSTEM

- Many teams write quick scripts to get started.
- Over time, the pipeline becomes fragile, hard to maintain and difficult to debug.
- It lacks proper structure, monitoring, security and ownership.

IMPACT

- Inconsistent behaviour
- Hard to troubleshoot
- Frequent failures
- Security gaps
- No scalability

2

BUILDING ONE GIANT MONOLITHIC PIPELINE

- All stages (build, test, release, deploy) are packed into one long pipeline.
- A failure in one step stops everything.
- It becomes slow, hard to maintain and difficult to reuse across services.

IMPACT

- Long execution time
- Large blast radius
- Difficult to debug
- Hard to make changes
- Low developer productivity

3

MIXING BUILD, TEST, RELEASE AND DEPLOYMENT RESPONSIBILITIES

- Build, test, release and deploy have different goals and risk levels.
- Mixing them creates confusion, weak controls and unintended deployments.
- It becomes harder to apply environment-specific policies and approvals.

IMPACT

- Unclear ownership
- Higher deployment risk
- Difficult to apply controls
- Increased chance of human error
- Harder to scale

KEY FOCUS AREAS



PIPELINE ARCHITECTURE

Design clear, modular and scalable pipelines.



STAGE BOUNDARIES

Separate build, test, release and deploy.



FAILURE ISOLATION

A failure in one stage should not break everything.



MAINTAINABILITY

Keep pipelines simple, modular and reusable.



BLAST RADIUS

Reduce the impact of failures through isolation.



CLEAR OWNERSHIP

Define who owns and maintains each part.

GOOD PIPELINE DESIGN (EXAMPLE)



BUILD

- Compile
- Build artifact
- Scan dependencies



TEST

- Unit tests
- Integration tests
- Security tests
- Quality checks



RELEASE

- Package
- Version
- Sign artifact
- Push to registry



DEPLOY

- Deploy to environment
- Health checks
- Monitoring
- Rollback plan



SENIOR ENGINEER INSIGHT

"A pipeline that is difficult to reason about during failure is already an operational risk."



PRODUCTION TIP

Design your pipeline like a production system: modular, observable, secure, maintainable and easy to debug. Future you will thank you.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**BUILD SKILLS
BUILD OPPORTUNITIES**

NON-REPRODUCIBLE BUILDS

3 COMMON MISTAKES AND HOW TO FIX THEM

4

USING FLOATING DEPENDENCY VERSIONS

- Using latest or unpinned versions (e.g. express:latest)
- Dependencies can change without notice
- Same code can produce different artifacts at different times

IMPACT

- Different builds
- Unexpected failures
- Security vulnerabilities
- Hard to debug
- No deterministic releases

5

BUILDING DIFFERENTLY ACROSS ENVIRONMENTS

- Using different tools, versions or configurations in local, CI and production builds
- Environment-specific dependencies or settings
- Builds behave differently, producing different artifacts

IMPACT

- Inconsistent artifacts
- Environment-specific bugs
- Works on my machine issues
- Difficult troubleshooting
- Loss of trust in the pipeline

6

DEPENDING ON MUTABLE BUILD ENVIRONMENTS

- Using shared or long-lived build runners
- Tools and dependencies change over time
- No isolation, builds are affected by previous jobs or manual changes

IMPACT

- Unpredictable build results
- Hidden dependencies
- Build failures after environment changes
- Difficult to reproduce or debug
- Security and compliance risks

HOW TO FIX THESE MISTAKES

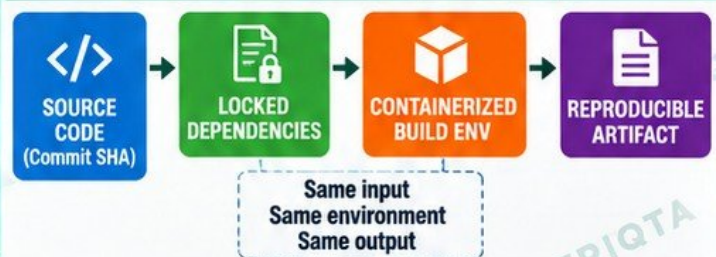
- ✓ **DEPENDENCY PINNING**
Use exact versions (e.g. version numbers, commit SHAs).
- ✓ **LOCK FILES**
Use package-lock.json, requirements.txt, go.sum, etc.
- ✓ **IMMUTABLE BUILD ENVIRONMENTS**
Use containers or ephemeral runners with fixed tool versions.
- ✓ **HERMETIC BUILDS**
Ensure the build does not rely on external or changing state.
- ✓ **REPRODUCIBILITY**
The same code always produces the same artifact.
- ✓ **BUILD PROVENANCE**
Track what was built, with what, and from which commit (e.g. SBOM, build metadata).



SENIOR ENGINEER INSIGHT

"If you cannot reproduce your builds, you cannot trust your deployments. Reproducibility is a foundation for reliability, security and speed."

REPRODUCIBLE BUILD FLOW (EXAMPLE)



PRODUCTION SCENARIO

Commit A builds successfully Monday but produces a different artifact Friday. Investigate why.



THINGS TO CHECK

- Are you using floating dependency versions?
- Did a dependency release a new version?
- Are build tools or base images updated?
- Are environment variables or build arguments different?
- Is the build runner mutable (shared/long-lived)?
- Compare artifact checksums and build logs.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

BUILDING THE SAME ARTIFACT MULTIPLE TIMES

MISTAKES 7-8 AND HOW TO FIX THEM

7

REBUILDING APPLICATION ARTIFACTS FOR EVERY ENVIRONMENT

- Building separate artifacts for dev, staging and production.
- Creates inconsistency and hard to track versions.
- Increases build time, cost and risk of environment drift.

IMPACT

- Different artifacts across environments
- Harder to debug issues
- Inconsistent behaviour in production
- Loss of traceability
- Higher cost and longer pipelines

8

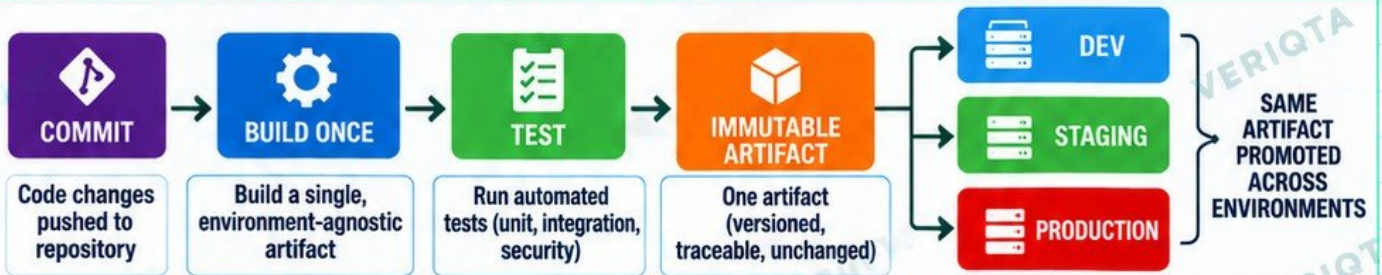
ALLOWING ENVIRONMENT-SPECIFIC CHANGES INSIDE THE ARTIFACT

- Embedding environment configuration in the artifact (e.g. config files, API endpoints, secrets).
- Rebuilding the application just to change environment values.
- Leads to configuration drift and increases the risk of human error.

IMPACT

- Artifacts behave differently
- More chances of misconfiguration
- Harder to promote between environments
- Security risks (e.g. secrets in artifacts)
- Breaks immutability and auditability

CORRECT MODEL: BUILD ONCE, PROMOTE MANY



KEY LESSONS

- ✓ **BUILD ONCE, PROMOTE MANY**
Create one artifact and deploy the same version across environments.
- ✓ **ARTIFACT IMMUTABILITY**
Artifacts must not change after they are built.
- ✓ **ENVIRONMENT CONFIGURATION**
Use external configuration (e.g. environment variables, config maps, secrets) instead of baking values into the artifact.
- ✓ **ARTIFACT INTEGRITY**
Version, sign and store artifacts in a registry. Ensure traceability from commit to production.



SENIOR ENGINEER INSIGHT

"The same code should behave the same way everywhere. If you have to rebuild to change an environment value, your pipeline design is already broken."



REAL-WORLD EXAMPLE

A team built separate artifacts for dev and production. A critical bug was fixed in the dev build but not in the production build, causing an incident. The rebuild process hid the inconsistency.



BEST PRACTICE

Keep your application artifact environment-agnostic. Inject environment-specific configuration at deploy time using secure and centralized methods.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**BUILD SKILLS
BUILD OPPORTUNITIES**

WEAK ARTIFACT MANAGEMENT

MISTAKES 9-11 AND HOW TO FIX THEM

9

OVERWRITING ARTIFACT VERSIONS

- Using the same version tag (e.g. latest)
- Overwriting artifacts in the registry
- Losing track of what was deployed

10

NOT RETAINING PREVIOUS DEPLOYABLE RELEASES

- Keeping only the latest artifact
- Unable to roll back quickly
- Losing older versions that were stable

11

DEPLOYING ARTIFACTS WITHOUT PROVENANCE

- No link between artifact and source code
- Unknown build environment
- Cannot verify integrity or trust the artifact
- Higher risk of supply chain attacks

WHAT GOOD ARTIFACT MANAGEMENT LOOKS LIKE



KEY PRACTICES

- ✓ **IMMUTABLE VERSIONING**
Use unique version numbers (e.g. v1.0.0) never overwrite artifacts.
- ✓ **COMMIT SHA CORRELATION**
Link every artifact to a specific commit SHA.
- ✓ **ARTIFACT REGISTRIES**
Use reliable registries (e.g. Docker Hub, ECR, GCR, Nexus, Artifactory).
- ✓ **CHECKSUMS AND SIGNATURES**
Verify artifact integrity using checksums and digital signatures.
- ✓ **SBOMS (SOFTWARE BILL OF MATERIALS)**
Generate an SBOM to track dependencies and detect vulnerabilities.
- ✓ **PROVENANCE**
Record build metadata (who, what, when, where, how) using build provenance tools (e.g. SLSA).
- ✓ **ROLLBACK AVAILABILITY**
Retain previous versions and test rollback procedures regularly.

EXAMPLE: VERSIONED ARTIFACTS IN A REGISTRY

```
# Build and tag with version and commit SHA
docker build -t myapp:1.0.0 -t myapp:1.0.0-abc123 .
docker push myapp:1.0.0
docker push myapp:1.0.0-abc123

# Verify artifact digest
docker inspect myapp:1.0.0 --format='{{.Id}}'
```



PRODUCTION SCENARIO

Commit A builds successfully Monday but produces a different artifact Friday.

- Why? Dependencies changed, base image updated, or build environment is mutable.
- How to investigate: Compare commit SHA, build logs, base images, dependency versions and artifact checksums.
- Fix: Use immutable versions, lock files and consistent build environments.



SENIOR ENGINEER INSIGHT

"If you cannot reproduce, you cannot trust, and if you cannot trust, you cannot operate at scale."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**BUILD SKILLS
BUILD OPPORTUNITIES**

SECRETS LEAKING THROUGH CI/CD

MISTAKES 12-14 AND HOW TO FIX THEM

12 STORING SECRETS IN REPOSITORY FILES

- Hardcoding API keys, passwords or tokens in code.
- Secrets committed to Git history.
- Secrets left in config files, environment files or manifests.

13 PRINTING CREDENTIALS INTO PIPELINE LOGS

- Using commands that echo secrets.
- Running tools with debug mode that expose sensitive data.
- Secrets appear in logs and can be accessed by many people.

14 USING LONG-LIVED CI/CD CREDENTIALS

- Using permanent cloud keys or tokens.
- Credentials never rotated.
- Overly broad permissions.
- High risk if the credentials are leaked.

EXAMPLE: WHAT NOT TO DO

```
# Storing a secret in a file (BAD)
AWS_ACCESS_KEY_ID=AKIA...123
AWS_SECRET_ACCESS_KEY=abcd...xyz
```



```
# Printing a secret in logs (BAD)
echo "Deploying with token: $TOKEN"
```



```
# Using long-lived credentials (BAD)
aws configure set aws_access_key_id AKIA...
aws configure set aws_secret_access_key abcd...
```



EXAMPLE: A SAFER APPROACH

```
# Use a secret manager (example: AWS Secrets Manager)
aws secretsmanager get-secret-value \
--secret-id myapp/prod/db --query SecretString
```



```
# Do not print secrets (masking is enabled by CI/CD)
echo "Deploying application..."
```



```
# Use OIDC and short-lived credentials
aws sts assume-role-with-web-identity \
--role-arn arn:aws:iam::123456789012:role/DeployRole \
--role-session-name github-actions
```



HOW TO FIX THESE MISTAKES



USE A SECRET MANAGER

Store secrets in services like AWS Secrets Manager, Azure Key Vault or HashiCorp Vault.



ENABLE SECRET MASKING

Ensure CI/CD masks secrets in logs automatically.



USE WORKLOAD IDENTITY

Use OIDC federation instead of static credentials.



USE SHORT-LIVED CREDENTIALS

Generate time-limited tokens with minimal scope.



ROTATE CREDENTIALS

Implement automatic credential rotation.



APPLY LEAST PRIVILEGE

Give only the permissions required for the job.



PRODUCTION FAILURE

A supposedly private pipeline log becomes a credential exposure path.

Real incident:

- A developer enabled debug mode in a pipeline.
- A cloud token was printed in the logs.
- The logs were accessible to all team members.
- The token was used by an attacker to access cloud resources.
- Result: data breach and costly incident.



KEY TAKEAWAYS

- Treat secrets as production data.
- Never store secrets in the repository.
- Use modern authentication (OIDC, workload identity).
- Keep credentials short-lived and rotated.
- Monitor for accidental secret exposure.



SENIOR ENGINEER INSIGHT

"If your CI/CD pipeline can leak a secret, it will eventually leak a secret."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

EXCESSIVE PIPELINE PERMISSIONS

MISTAKES 15-16 AND HOW TO FIX THEM

15

GIVING CI/CD ADMINISTRATIVE CLOUD PERMISSIONS

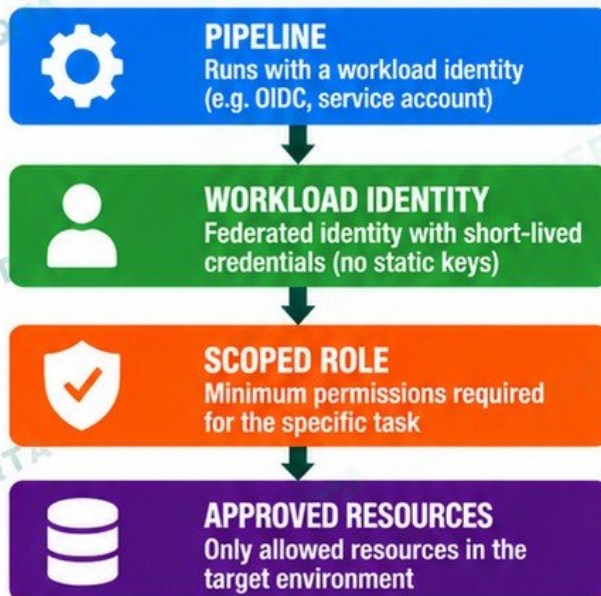
- Using admin or overly broad roles (e.g. AdministratorAccess).
- CI/CD can create, delete or modify any resource.
- Increases the blast radius if credentials are compromised.

16

SHARING DEPLOYMENT IDENTITIES ACROSS APPLICATIONS

- Same identity used for multiple applications or environments.
- No isolation between teams and services.
- A compromise in one application can affect all other applications.

CORRECT MODEL: LEAST PRIVILEGE PIPELINE



KEY CONTROLS



IAM BOUNDARIES

Restrict what the pipeline can access.



PER-SERVICE IDENTITIES

Use a separate identity for each application.



ENVIRONMENT ISOLATION

Different identities for dev, staging and production.



PRIVILEGE ESCALATION RISK

Prevent the pipeline from assuming unnecessary roles.



AUDITABILITY

Track every action made by the pipeline identity.

REAL-WORLD EXAMPLE (BAD)

```
# GitHub Actions with admin permissions (BAD)
permissions:
  id-token: write
  contents: read
env:
  AWS_ROLE_ARN: arn:aws:iam::123456789012:role/Admin
```



BETTER APPROACH

```
# GitHub Actions with scoped role (GOOD)
permissions:
  id-token: write
  contents: read
env:
  AWS_ROLE_ARN: arn:aws:iam::123456789012:role/
  deploy-myapp-prod # Minimal permissions
```



SENIOR ENGINEER INSIGHT

"Your CI/CD pipeline is production infrastructure. Treat its permissions with the same care as your production systems."



PRODUCTION TIP

Use OIDC federation or workload identity instead of long-lived static credentials. Grant only the permissions needed for the specific deployment.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**BUILD SKILLS
BUILD OPPORTUNITIES**

UNSAFE THIRD-PARTY ACTIONS AND DEPENDENCIES

MISTAKES 17-19 AND HOW TO FIX THEM

17 USING UNPINNED THIRD-PARTY PIPELINE ACTIONS

- Using actions by tag (e.g. v1, v2, latest).
- Tags can be moved to different code.
- Attackers can inject malicious changes.
- Your pipeline can run untrusted code.

18 TRUSTING EXTERNAL BUILD DEPENDENCIES BLINDLY

- Using untrusted packages from public repositories.
- Not verifying the source or integrity.
- Allowing automatic dependency updates.
- Introducing vulnerable or malicious dependencies.

19 RUNNING UNTRUSTED CODE WITH PRIVILEGED CREDENTIALS

- Giving third-party actions excessive permissions.
- Running untrusted code with cloud credentials.
- Exposing secrets to external code.
- A compromised action can access your infrastructure.

SUPPLY-CHAIN ATTACK FLOW



KEY FOCUS AREAS

- SUPPLY-CHAIN ATTACKS**
Understand the risk from third-party code.
- COMMIT SHA PINNING**
Pin actions and dependencies to a specific commit SHA.
- DEPENDENCY REVIEW**
Review and audit external dependencies regularly.
- RUNNER ISOLATION**
Use isolated and least-privileged runners.
- TRUSTED PUBLISHERS**
Use actions from verified and trusted sources.
- ALLOWLISTING**
Allow only approved actions and dependencies.

BAD vs GOOD EXAMPLE (GitHub Actions)

✗ BAD (UNPINNED)

uses: actions/checkout@v3

- Uses a moving tag (v3)
- Can be changed at any time
- Higher risk of malicious code

✓ GOOD (PINNED)

uses: actions/checkout@3df4e2a7c9b1e8f6d...

- Uses a specific commit SHA
- Immutable and predictable
- Much safer

ADDITIONAL BEST PRACTICES

- ✓ Verify checksums for downloaded packages.
- ✓ Use private registries for critical dependencies.
- ✓ Scan dependencies for known vulnerabilities.
- ✓ Use minimal permissions for pipeline jobs.
- ✓ Regularly review and update allowed actions.
- ✓ Monitor for suspicious changes in dependencies.



REAL-WORLD EXAMPLE

A popular third-party action was compromised, allowing attackers to steal CI/CD secrets from thousands of repositories. Teams that pinned to a commit SHA were not affected.



SENIOR ENGINEER INSIGHT

"Every external dependency is a potential entry point. Trust, but always verify."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

TESTING TOO LATE

MISTAKES 20-22 AND HOW TO FIX THEM

20

RUNNING CHEAP TESTS AFTER EXPENSIVE TESTS

- Running integration or E2E tests before lint and unit tests.
- Wastes time and compute resources.
- Simple issues could have been caught much earlier.

21

DISCOVERING FAILURES NEAR DEPLOYMENT

- Finding critical issues late in the pipeline.
- Increases cost, delays releases and creates last-minute pressure.
- Higher risk of failed deployments.

22

TREATING EVERY TEST AS EQUALLY VALUABLE

- Running all tests on every change without prioritization.
- Slows down the pipeline.
- Does not consider the different cost, speed and value of each test type.

THE OPTIMIZED FEEDBACK PATH



WHY THIS ORDER MATTERS

- ✓ Finds issues early when they are cheaper to fix.
- ✓ Reduces wasted compute and pipeline time.
- ✓ Prevents low-quality builds from reaching later stages.
- ✓ Improves developer productivity and confidence.
- ✓ Creates a faster and more reliable release process.

KEY DISCUSSION POINTS

- FEEDBACK LATENCY** How quickly do developers get feedback?
- TEST COST** What is the compute and time cost of each test?
- FAILURE PROBABILITY** Which tests are most likely to catch real issues?
- PIPELINE DURATION** How does test ordering affect total pipeline time?
- PARALLELIZATION** Which tests can run in parallel to save time?

REAL-WORLD EXAMPLE (BAD)

- ✗ A team runs E2E tests first. The E2E test fails due to a simple code style issue that could have been caught by a linter in seconds. Result: 30 minutes of wasted pipeline time.

BETTER APPROACH

- ✓ Run lint and unit tests first. Catch simple issues early, save time and keep the pipeline fast and reliable.



SENIOR ENGINEER INSIGHT

"The value of a test is not in how complex it is, but in how early it can prevent a bigger failure."

THE FLAKY TEST PROBLEM

MISTAKES 23-24 AND HOW TO FIX THEM

23 IGNORING FLAKY TESTS

- Treating flaky tests as a minor issue.
- Allowing unstable tests to remain in the pipeline.
- Reduces trust in test results.
- Can hide real product defects.

24 AUTOMATICALLY RERUNNING FAILURES UNTIL THEY PASS

- Using automatic retries to force a pass.
- Creates false confidence.
- Hides real failures.
- Increases mean time to detect real issues.
- Wastes compute resources.

PRODUCTION CONSEQUENCE



REAL FAILURE

A test fails due to a real issue (code, environment or dependency).



RETRY

The pipeline automatically reruns the test.



PASS

The test passes on retry (temporary, non-deterministic).



FAILURE HIDDEN

The real issue is hidden and can reach production.

KEY CONCEPTS TO UNDERSTAND



FLAKE RATE

The percentage of tests that fail intermittently without code changes.



TEST QUARANTINE

Isolate flaky tests so they do not block merges while you investigate and fix them.



ROOT-CAUSE OWNERSHIP

Assign ownership to investigate and fix flaky tests. Do not ignore them.



DETERMINISTIC ENVIRONMENTS

Use consistent, isolated and reproducible environments (same OS, dependencies, data).



FAILURE EVIDENCE

Capture logs, screenshots, artifacts and traces to understand the real cause.

BEST PRACTICES

- ✓ Track and monitor flake rate.
- ✓ Quarantine and fix flaky tests.
- ✓ Use deterministic test environments.
- ✓ Avoid automatic retries.
- ✓ Investigate and document root cause.
- ✓ Set an acceptable flake rate threshold (e.g. $< 1\%$).



REAL-WORLD EXAMPLE

A payment service had a flaky test that failed intermittently due to a race condition. The pipeline retried the test until it passed. The real issue reached production and caused failed transactions for users.



SENIOR ENGINEER INSIGHT

"A flaky test is a liability. If you do not fix it, it will eventually hide a real failure when it matters most."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

SLOW PIPELINES

MISTAKES 25-27 AND HOW TO FIX THEM

25 RUNNING EVERY JOB SEQUENTIALLY

- Running all jobs one after another.
- Wastes time and resources.
- Does not take advantage of parallel execution.
- Increases feedback time and developer frustration.

26 CACHING WITHOUT UNDERSTANDING INVALIDATION

- Using caches but not understanding when they should be invalidated.
- Leads to stale dependencies and build issues.
- Can cause inconsistent and unreliable builds.
- May hide real problems.

27 RUNNING UNNECESSARY WORK FOR EVERY CHANGE

- Running the full pipeline even for small changes.
- Not using path filters or conditional execution.
- Wastes compute resources.
- Slows down feedback loop.
- Increases cloud costs.

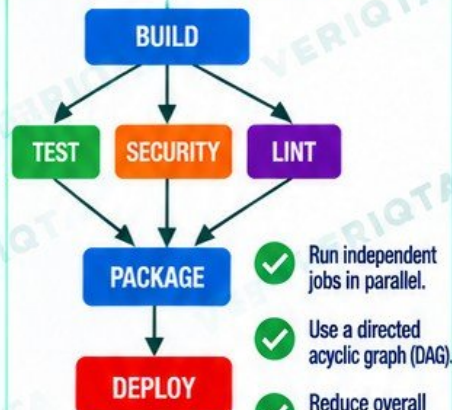
FROM SEQUENTIAL TO PARALLEL (DAG)

SEQUENTIAL (SLOW)



Total time: 30 minutes

DAG-BASED (FASTER)



- ✓ Run independent jobs in parallel.
- ✓ Use a directed acyclic graph (DAG).
- ✓ Reduce overall pipeline time.

Total time: 10 minutes

KEY TECHNIQUES TO SPEED UP PIPELINES



DAG-BASED EXECUTION

Define job dependencies and run independent jobs in parallel.



PARALLELISM

Run multiple jobs simultaneously.



SELECTIVE EXECUTION

Run only what is needed (e.g. by branch, path or change type).



PATH FILTERS

Trigger jobs only when specific files or directories change.



DEPENDENCY CACHING

Cache dependencies to avoid repeated downloads.



CACHE INVALIDATION

Understand when to invalidate caches (e.g. lock file changes, version updates).



CRITICAL PATH ANALYSIS

Identify and optimize the slowest parts of the pipeline.

OBSERVABILITY: KEY METRICS TO TRACK



QUEUE TIME

Time spent waiting for a runner to become available.



EXECUTION TIME

Time taken to run each job and the entire pipeline.



CACHE HIT RATE

Percentage of cache hits vs misses.



STAGE DURATION

Time taken by each stage (build, test, deploy, etc.).



FAILURE RATE

How often jobs or the pipeline fail.



REAL-WORLD EXAMPLE

A team ran the full test suite for every commit. Even documentation changes took 25 minutes. By using path filters and parallel jobs, they reduced pipeline time by 70% and cut costs significantly.



SENIOR ENGINEER INSIGHT

"A fast pipeline is not a luxury. It is a force multiplier for the entire team."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

RUNNER AND AGENT MISTAKES

MISTAKES 28–30 AND HOW TO FIX THEM

28 REUSING DIRTY BUILD RUNNERS

- Reusing runners without cleaning the workspace.
- Leftover files, dependencies or credentials can affect new builds.
- Can cause unpredictable build results and security risks.

29 ALLOWING UNCONTROLLED PRIVILEGED RUNNERS

- Giving runners unnecessary privileged access (e.g. root, docker, cloud credentials).
- Can be exploited to gain access to your infrastructure.
- Increases the impact of compromised builds.

30 IGNORING RUNNER CAPACITY AND QUEUE SATURATION

- Not monitoring runner capacity and queue depth.
- Leads to long wait times and slower feedback.
- Can block critical releases.
- Wastes developer time and affects productivity.

KEY CONCEPTS AND BEST PRACTICES



EPHEMERAL RUNNERS

Use short-lived runners that are created fresh for each job and destroyed after completion.



ISOLATION

Keep builds isolated to prevent cross-project contamination and security risks.



AUTOSCALING

Automatically scale runners based on demand to handle peak workloads.



WORKSPACE CONTAMINATION

Always clean workspaces or use ephemeral runners to avoid leftover files and caches.



PRIVILEGED EXECUTION

Avoid unnecessary privileged access. Use the minimum permissions required.



QUEUE DEPTH

Monitor the number of pending jobs and set alerts for high queue times.

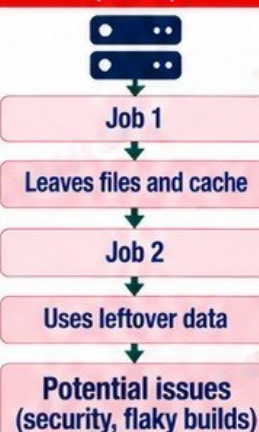


RUNNER EXHAUSTION

Ensure enough runners are available to prevent bottlenecks and failed jobs.

RUNNER LIFECYCLE COMPARISON

PERSISTENT RUNNER (RISKY)



EPHEMERAL RUNNER (RECOMMENDED)



OBSERVABILITY METRICS



QUEUE TIME

Time jobs wait for a runner.



EXECUTION TIME

Time taken to complete the job.



RUNNER UTILIZATION

Percentage of runner capacity used.



QUEUE DEPTH

Number of pending jobs.



FAILURE RATE

Percentage of failed jobs due to runner issues.

REAL-WORLD EXAMPLE



A company reused self-hosted runners without cleaning the workspace. A leftover file from a previous project caused tests to pass incorrectly, leading to a broken release in production.

HOW TO FIX IT



- ✓ Use ephemeral runners.
- ✓ Implement autoscaling.
- ✓ Monitor queue depth.
- ✓ Keep runners isolated.
- ✓ Grant minimum privileges.



SENIOR ENGINEER INSIGHT

“Your CI/CD runners are part of your production security boundary. Treat them like servers, not shared machines.”



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

DEPLOYMENT WITHOUT SAFETY CONTROLS

MISTAKES 31–33 AND HOW TO FIX THEM

31 DEPLOYING DIRECTLY TO ALL PRODUCTION INSTANCES

- Releasing new versions to every instance at once.
- No rollback window if issues occur.
- Increases blast radius and user impact.
- Can cause full outages.

32 HAVING NO PROGRESSIVE DELIVERY MECHANISM

- No canary, blue-green or rolling deployment.
- Cannot detect issues early with a small user group.
- Misses the opportunity to validate in production.
- Leads to higher risk releases.

33 TREATING DEPLOYMENT SUCCESS AS APPLICATION SUCCESS

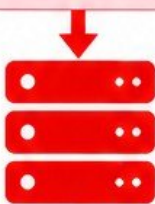
- A successful deploy does not mean the application is healthy.
- Deployment can succeed while the app is broken.
- Not verifying real user functionality.
- Ignores monitoring and health checks.

DEPLOYMENT APPROACH COMPARISON

BIG BANG DEPLOYMENT

High risk. All users affected at once.

100% TRAFFIC

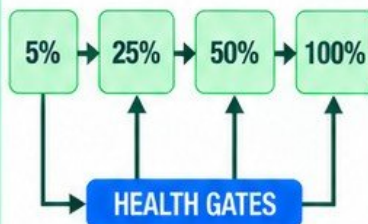


NEW VERSION

- ✗ All instances updated at once
- ✗ High risk of widespread failure
- ✗ No progressive validation
- ✗ Harder and slower to rollback

CANARY DEPLOYMENT

Low risk. Gradual traffic increase.



- ✓ Small percentage of users first
- ✓ Validate with real traffic
- ✓ Automatically move to next stage if healthy
- ✓ Quick rollback if issues detected
- ✓ Reduces risk and user impact

OTHER DEPLOYMENT STRATEGIES



CANARY DEPLOYMENT

Release to a small percentage of users, then gradually increase traffic based on health metrics.



BLUE-GREEN DEPLOYMENT

Run two identical environments (blue and green). Switch traffic once the new version is verified.



ROLLING DEPLOYMENT

Update instances in batches instead of all at once.

KEY SAFETY CONTROLS



HEALTH GATES

Automatically verify application health at each stage.



AUTOMATED VERIFICATION

Run smoke tests, synthetic tests and real user checks.



MONITORING

Track error rates, latency, and key business metrics.



AUTOMATED ROLLBACK

Rollback automatically if health checks fail.

REAL-WORLD EXAMPLE



A company deployed a new version directly to all production instances. The deployment succeeded, but a critical configuration error caused high error rates. All users were affected, leading to lost revenue and service downtime. A canary deployment would have detected the issue with 5% of traffic.



SENIOR ENGINEER INSIGHT

“A safe deployment is not just about releasing new code. It is about proving that the application works for real users before everyone is affected.”



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

BROKEN ROLLBACK STRATEGIES

MISTAKES 34-36 AND HOW TO FIX THEM

34 ASSUMING EVERY DEPLOYMENT CAN SIMPLY BE ROLLED BACK

- Not all changes are easily reversible.
- May have data, schema or infrastructure changes.
- Can lead to longer downtime and higher risk.

35 NEVER TESTING ROLLBACK PROCEDURES

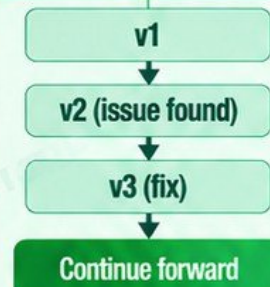
- Rollback may fail when needed most.
- Teams are not familiar with the process.
- Hidden issues can make recovery slower and harder.

36 IGNORING DATABASE COMPATIBILITY DURING ROLLBACK

- New schema changes may not be compatible with older application versions.
- Can cause application failures or data corruption.
- Requires backward-compatible design and careful planning.

ROLL-FORWARD VS ROLLBACK

ROLL FORWARD (RECOMMENDED IN MANY CASES)



- ✓ Used when rollback is risky or not possible.
- ✓ Applies a new version with a fix.
- ✓ Keeps data changes intact.
- ✓ Common in modern systems and databases.

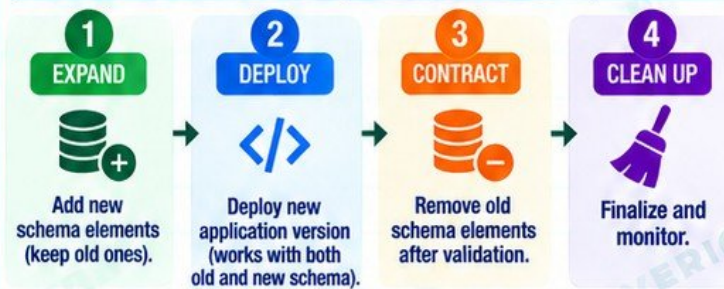
ROLLBACK (TRADITIONAL APPROACH)



- ✗ Only works if fully compatible.
- ✗ May not work after database changes.
- ✗ Can still leave data in an inconsistent state.
- ✗ Should be tested regularly.

DATABASE MIGRATION STRATEGY

EXPAND-CONTRACT PATTERN (SAFER DEPLOYMENTS)



This ensures backward compatibility and allows safe roll-forward without breaking older versions.

KEY CONCEPTS TO UNDERSTAND

- BACKWARD-COMPATIBLE SCHEMA CHANGES**
Design changes that work with both old and new versions.
- ROLL-FORWARD VS ROLLBACK**
Sometimes rolling forward is safer than rolling back.
- EXPAND-CONTRACT MIGRATIONS**
Add new elements first, remove old elements later.
- ROLLBACK TESTING**
Regularly test rollback procedures in a non-production environment.
- APPLICATION/DATABASE COUPLING**
Understand how tightly the application depends on the database.

REAL-WORLD EXAMPLE

A team deployed application v2 with a new database schema that changed the format of a user profile field. They later rolled back to v1, but v1 could not read the new data format. The application failed, causing a longer outage than if they had rolled forward with a proper fix.

SENIOR ENGINEER QUESTION

What happens when application v2 modifies data that application v1 cannot understand?

EXPECTED ANSWER:

Rollback may fail because v1 cannot parse or correctly handle the new data. In such cases, you may need to roll forward with a fixed version, use backward-compatible schema changes, or implement data transformation/migration strategies.

BEST PRACTICES

- ✓ Design backward-compatible schema changes.
- ✓ Use expand-contract migration patterns.
- ✓ Test rollback procedures regularly.
- ✓ Validate application and database compatibility.
- ✓ Have a roll-forward plan.
- ✓ Automate verification after deployment.



SENIOR ENGINEER INSIGHT

“A good deployment strategy assumes failure and is tested for it.”

DATABASE CHANGES THAT BREAK RELEASES

MISTAKES 37–38 AND HOW TO FIX THEM

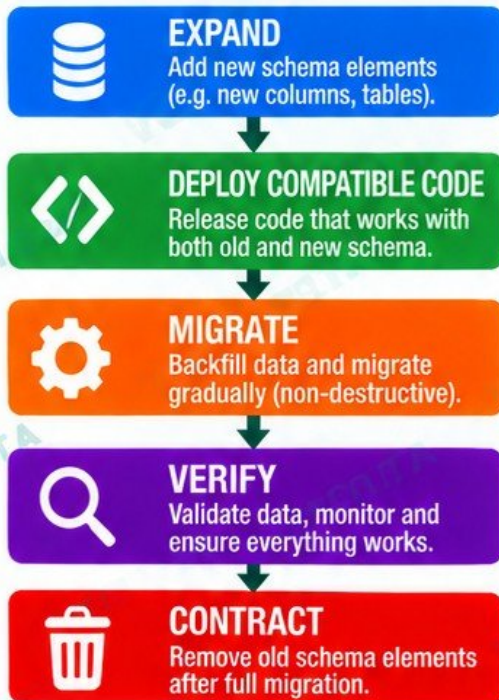
37 COUPLING DESTRUCTIVE MIGRATIONS DIRECTLY TO DEPLOYMENT

- Dropping or renaming columns while deploying new code.
- Causes application failures and downtime.
- Leaves no time for validation or recovery.
- Can result in data loss.

38 DEPLOYING APPLICATION AND SCHEMA CHANGES WITHOUT COMPATIBILITY PLANNING

- Releasing code that depends on schema changes that are not yet applied.
- Application and database become incompatible.
- Can cause runtime errors and failed deployments.
- Particularly risky in distributed systems with multiple services.

THE SAFE DATABASE CHANGE FLOW



KEY CONCEPTS

- ZERO-DOWNTIME MIGRATIONS**
Keep the application available during changes.
- SCHEMA COMPATIBILITY**
Ensure old and new versions work together.
- LONG-RUNNING MIGRATIONS**
Handle large datasets without blocking the system.
- LOCK CONTENTION**
Avoid operations that lock tables for long periods.
- RECOVERY**
Have a plan to recover if something goes wrong.

COMMON PROBLEMS

- Table locks can cause application downtime.
- Large migrations can run for hours or days.
- Dropping columns too early breaks older application versions.
- Application and database dependencies can be tightly coupled.
- Harder to roll back once data has been modified.

REAL-WORLD EXAMPLE

- A team deployed a new version that renamed a column and dropped the old one at the same time. Some application instances were still using the old column, causing errors and a complete outage. Proper expand-contract migration would have prevented this.

BEST PRACTICES

- ✓ Use the expand-contract pattern.
- ✓ Keep schema changes backward-compatible.
- ✓ Test migrations in a staging environment.
- ✓ Monitor for performance impact.
- ✓ Plan for rollback or roll-forward.
- ✓ Communicate changes across teams.

SENIOR ENGINEER INSIGHT

- “Database changes are not just a technical task. They are a production risk. Design for zero downtime, plan for compatibility, and always have a recovery strategy.”

MISSING DEPLOYMENT OBSERVABILITY

MISTAKES 39–40 AND HOW TO FIX THEM

39 NOT CORRELATING DEPLOYMENTS WITH PRODUCTION TELEMETRY

- Deploying without monitoring the real impact.
- Not checking if errors, latency or user issues increased after deployment.
- Treating deployment as “done” without verifying production health.
- Missing early signs of failure.

40 DECLARING SUCCESS BECAUSE THE PIPELINE RETURNED GREEN

- A green pipeline only means the deployment steps completed.
- It does not mean the application is working correctly in production.
- Real issues can appear minutes or hours after deployment.
- Can lead to undetected failures and user impact.

OBSERVE AFTER DEPLOYMENT

Monitor these key signals to understand the real impact of your deployment.



REAL-WORLD EXAMPLE

-  A team deployed a new version. The pipeline was green, so they considered it a success. 30 minutes later, error rate increased by 300% and latency doubled. Users could not complete purchases. The issue was only detected because production monitoring alerted on the anomaly.

BEST PRACTICES

- ✓ Set up dashboards for key deployment metrics.
- ✓ Compare metrics before and after deployment.
- ✓ Use alerts for abnormal changes.
- ✓ Monitor business-critical user journeys.
- ✓ Keep monitoring for hours after deployment.

DEPLOYMENT OBSERVABILITY FLOW



PRODUCTION LESSON

-  **“Green pipeline ≠ healthy production system.”**
Always verify production metrics, not just pipeline status.

SENIOR ENGINEER INSIGHT

-  **“A successful deployment is not when the pipeline finishes. It is when users continue to have a great experience.”**

BAD APPROVAL GATES

MISTAKES 41–42 AND HOW TO FIX THEM

41 REQUIRING MANUAL APPROVAL FOR EVERYTHING

- Slows down delivery and reduces team productivity.
- Creates unnecessary bottlenecks.
- Leads to approval fatigue, where important approvals are ignored.
- Increases time to market.
- Teams find ways to bypass the process.

42 HAVING NO CONTROL FOR GENUINELY HIGH-RISK CHANGES

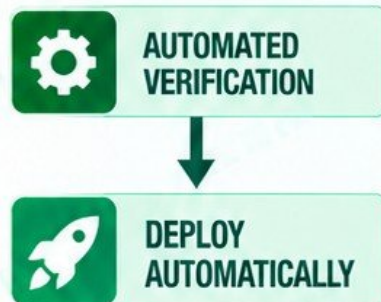
- Allows risky changes to go to production without proper review.
- Can lead to security, compliance and business risks.
- No separation of duties.
- Difficult to meet regulatory requirements.
- Higher chance of outages and data loss.

RISK-BASED DEPLOYMENT CONTROLS

Different changes require different levels of control.

LOW RISK

Small, non-breaking changes (e.g. docs, UI tweaks, low impact).



- ✓ Fast and efficient
- ✓ No manual approval needed
- ✓ Continuous delivery enabled
- ✓ Low operational risk

HIGH RISK

Major changes (e.g. infrastructure, security, database, core services).



- ✓ Controlled and deliberate
- ✓ Proper review and validation
- ✓ Reduced risk of failure
- ✓ Complies with security and regulatory requirements

KEY CONCEPTS



CHANGE RISK

Assess the potential impact of each change.



SEPARATION OF DUTIES

Ensure different people approve, implement and verify changes.



COMPLIANCE

Meet regulatory and internal policy requirements.



AUTOMATION

Automate low-risk changes to improve velocity.



DEPLOYMENT VELOCITY

Balance speed with safety and risk control.



EMERGENCY PROCEDURES

Have a fast and clear process for urgent fixes (break-glass).

REAL-WORLD EXAMPLE



A company required manual approval for all deployments, including small documentation changes. This created a 3-day delay for every release. At the same time, a high-risk database change was deployed without proper review, causing a major outage. A risk-based approach would have prevented both problems.

BEST PRACTICES

- ✓ Classify changes by risk level.
- ✓ Automate approvals for low-risk changes.
- ✓ Require approval for high-risk changes.
- ✓ Use progressive delivery for risky releases.
- ✓ Maintain audit logs of all approvals.
- ✓ Have a clear emergency change process.

SENIOR ENGINEER INSIGHT



"The goal is not more approvals. The goal is the right approvals. Fast and safe delivery is possible when you use risk-based controls."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

ENVIRONMENT DRIFT

MISTAKES 43-44 AND HOW TO FIX THEM

43

MANUALLY CHANGING INFRASTRUCTURE OUTSIDE AUTOMATION

- Making direct changes in production (SSH, console, UI).
- Changes are not tracked or versioned.
- Creates configuration drift.
- Hard to reproduce and troubleshoot.
- Can be lost or overwritten by future deployments.

44

ALLOWING STAGING AND PRODUCTION ARCHITECTURE TO DIVERGE UNPREDICTABLY

- Environments have different configurations, versions or resources.
- Works in staging but fails in production.
- Makes testing unreliable.
- Increases mean time to resolution (MTTR).
- Leads to unexpected behavior and outages.

THE PROBLEM: ENVIRONMENT DRIFT

IDEAL STATE



SAME ARCHITECTURE
SAME CONFIGURATION
PREDICTABLE BEHAVIOR

DRIFTED STATE



DIFFERENT CONFIGURATIONS
UNPREDICTABLE BEHAVIOR
HIGHER RISK OF FAILURE

HOW TO PREVENT ENVIRONMENT DRIFT



INFRASTRUCTURE AS CODE

Define and manage infrastructure using code (Terraform, CloudFormation, etc.).



CONFIGURATION MANAGEMENT

Use tools like Ansible, Chef, Puppet to enforce configuration.



DRIFT DETECTION

Continuously detect and alert on out-of-band changes.



IMMUTABLE INFRASTRUCTURE

Replace, don't modify infrastructure.



ENVIRONMENT PARITY

Keep environments as similar as possible (same versions, configurations, dependencies).



CONTROLLED DIFFERENCES

Document and intentionally manage necessary differences (e.g. smaller size, dummy data).

TOOLS FOR DRIFT DETECTION



Terraform drift detection
terraform plan



AWS Config

Track configuration changes



Google Cloud Config

Monitor resource changes



Azure Policy

Detect non-compliant changes



Configuration management

Ansible, Chef, Puppet



Monitoring and alerting

Datadog, Prometheus, Grafana

DEBUGGING QUESTION



Is the failure caused by the application or by an environment difference?

Always compare configurations, versions, and infrastructure between environments when troubleshooting.

REAL-WORLD EXAMPLE



A service worked perfectly in staging but failed in production. The production server had a manually installed package that was not in staging. This difference caused a runtime error. The issue was resolved by removing manual changes and using infrastructure as code to keep environments consistent.

BEST PRACTICES



- ✓ Use infrastructure as code for all environments.
- ✓ Avoid manual changes in production.
- ✓ Enable drift detection and alerts.
- ✓ Keep staging and production as similar as possible.
- ✓ Document and manage intentional differences.
- ✓ Regularly audit and reconcile environments.

SENIOR ENGINEER INSIGHT



"Environment drift is the enemy of reliability. If you can't reproduce it, you can't fix it. Consistency is a force multiplier."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

NO DEPLOYMENT CONCURRENCY CONTROL

MISTAKES 45–47 AND HOW TO FIX THEM

45 ALLOWING OVERLAPPING PRODUCTION DEPLOYMENTS

- Multiple pipelines deploy at the same time.
- Can cause unpredictable results.
- Hard to know which version is actually running.
- May overwrite a newer release with an older one.

46 DEPLOYING STALE COMMITS AFTER NEWER RELEASES

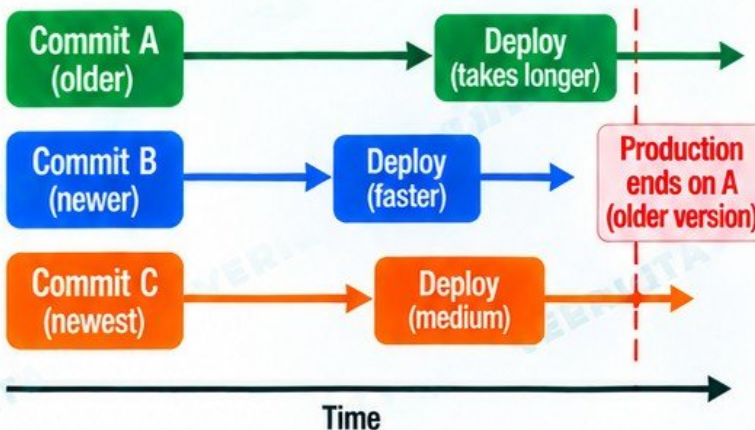
- Older commits are deployed after newer commits.
- Can roll back features unintentionally.
- Creates confusion and inconsistency.
- Users may see unexpected behavior.

47 IGNORING RACE CONDITIONS BETWEEN PIPELINES

- Multiple pipelines compete for the same resources.
- Can cause configuration drift or partial deployments.
- May lead to service instability or downtime.
- Hard to reproduce and troubleshoot.

SCENARIO: OVERLAPPING DEPLOYMENTS

Three commits are deployed, but they complete in the wrong order.



Even though B and C were newer, A finished last, so production ended up running the older version (A).

HOW TO FIX IT



DEPLOYMENT LOCKS

Allow only one production deployment at a time.



CONCURRENCY GROUPS

Ensure pipelines for the same environment do not run in parallel.



COMMIT ORDERING

Deploy commits in the correct order (newest only).



CANCELLATION

Automatically cancel in-progress runs when a newer commit is pushed.



DEPLOYMENT QUEUES

Use a queue to manage deployment requests.



RELEASE SERIALIZATION

Ensure releases are deployed sequentially and predictably.

REAL-WORLD EXAMPLE



A team pushed three commits (A, B, C) within 30 minutes. All three pipelines ran and deployed to production. Commit C finished first, then B, but A finished last. Production ended up running commit A, which did not have important security fixes. Users experienced issues until the team deployed the correct version again.

BEST PRACTICES

- ✓ Use concurrency controls for production deployments.
- ✓ Cancel outdated runs automatically.
- ✓ Deploy only the latest commit.
- ✓ Use deployment queues or locks.
- ✓ Monitor deployment status and order.
- ✓ Document and enforce a clear release process.

SENIOR ENGINEER INSIGHT



“A deployment is not done when it starts or when it finishes. It is done when the right version is running in production.”

Control the flow, or the flow will control you.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS
BUILD OPPORTUNITIES

CI/CD WITHOUT OWNERSHIP OR INCIDENT READINESS

MISTAKES 48-49 AND HOW TO FIX THEM

48 HAVING NO CLEAR OWNER FOR PIPELINE FAILURES

- No defined team or individual responsible for the CI/CD system.
- Issues can sit unresolved for hours or days.
- Unclear who to contact during failures.
- Leads to slow recovery and missed deployments.
- Creates frustration and finger-pointing between teams.

49 TREATING CI/CD OUTAGES AS A DEVELOPER INCONVENIENCE INSTEAD OF PRODUCTION RISK

- CI/CD is critical infrastructure, not a "nice to have".
- Outages block security fixes, feature releases and critical deployments.
- Can impact business revenue, customer trust and compliance.
- Should be treated with the same severity as production system outages.
- Requires proper monitoring, alerting and incident response.

WHAT GOOD OWNERSHIP LOOKS LIKE

-  **PLATFORM OWNERSHIP**
Assign a clear owner or team for CI/CD infrastructure, tooling and reliability.
-  **RUNBOOKS**
Document common failures, troubleshooting steps and recovery procedures.
-  **ESCALATION**
Define escalation paths, on-call rotation and communication channels.
-  **SLOs**
Set reliability targets (e.g. 99.9% pipeline availability) and track performance.
-  **INCIDENT RESPONSE**
Treat CI/CD outages as production incidents with defined severity and response process.
-  **DEPENDENCY MAPPING**
Understand dependencies (source control, registries, cloud services, runners, secret stores, etc.).
-  **RECOVERY PROCEDURES**
Have tested recovery steps and rollback plans for the CI/CD system itself.
-  **BREAK-GLASS ACCESS**
Maintain secure emergency access for critical situations when normal access is unavailable.

CI/CD INCIDENT FLOW



REAL-WORLD EXAMPLE

-  A company's CI/CD platform went down for 6 hours due to a misconfigured runner. There was no clear owner, so the issue was passed between teams. Critical security patches could not be deployed, leaving production vulnerable. Proper ownership, alerting and a runbook would have reduced the recovery time to minutes.

BEST PRACTICES

- ✓ Assign clear ownership for CI/CD.
- ✓ Maintain up-to-date runbooks.
- ✓ Set and monitor SLOs.
- ✓ Have a 24/7 on-call rotation.
- ✓ Test recovery procedures regularly.
- ✓ Treat CI/CD as production infrastructure.
- ✓ Review and improve after every incident.

SENIOR PERSPECTIVE

-  "If CI/CD controls production changes, CI/CD itself is production infrastructure. Treat it with the same seriousness, ownership and resilience."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**BUILD SKILLS
BUILD OPPORTUNITIES**

THE MOST DANGEROUS MISTAKE: NO FEEDBACK LOOP

MISTAKE 50 AND THE COMPLETE CI/CD MODEL

50

NEVER MEASURING WHETHER THE DELIVERY SYSTEM IS ACTUALLY IMPROVING

- No metrics to assess performance.
- No feedback loop to identify issues.
- Cannot tell if changes are making the system better or worse.
- Repeatedly make the same mistakes.
- Miss opportunities to optimize speed, stability and reliability.

WHY A FEEDBACK LOOP MATTERS

- IMPROVES RELIABILITY**
Find and fix issues early.
- INCREASES DEPLOYMENT VELOCITY**
Remove bottlenecks.
- DRIVES CONTINUOUS IMPROVEMENT**
Make data-driven decisions.
- REDUCES RISK**
Identify patterns before they cause major incidents.
- BUILDS CONFIDENCE**
Teams trust a system that shows real results.
- SUPPORTS BUSINESS GOALS**
Faster, safer and more stable delivery of value.

REAL-WORLD EXAMPLE

- ⚠️ A company automated deployments but never measured DORA metrics. Over time, pipeline duration increased, failures went unnoticed, and rollbacks became frequent. Without a feedback loop, the team did not realize the system was getting worse until a major production incident occurred.

FINAL SENIOR ENGINEERING MODEL



KEY METRICS TO MEASURE

- DEPLOYMENT FREQUENCY**
How often you deploy to production (e.g. per day or week).
- LEAD TIME**
Time from code commit to production.
- CHANGE FAILURE RATE**
Percentage of deployments that cause a failure.
- RECOVERY TIME**
Time to restore service after a failed deployment.
- PIPELINE FAILURE RATE**
Percentage of pipeline runs that fail.
- PIPELINE DURATION**
Total time for the pipeline to complete.
- QUEUE TIME**
Time a run waits before starting.
- ROLLBACK FREQUENCY**
How often deployments are rolled back.
- DEPLOYMENT-RELATED INCIDENTS**
Number and severity of incidents caused by deployments.
- SLO IMPACT**
How deployments affect your service level objectives (availability, latency, error rate, etc.).

BEST PRACTICES

- ✓ Track and review key metrics regularly.
- ✓ Set targets and alert on regressions.
- ✓ Use post-deployment reviews.
- ✓ Share metrics with the whole team.
- ✓ Continuously optimize the delivery process.
- ✓ Automate data collection and reporting.

FINAL PRINCIPLE

- 💡 "A mature CI/CD system does not merely automate deployment. It provides a controlled, observable, reproducible, secure, and recoverable path for changing production."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**BUILD SKILLS
BUILD OPPORTUNITIES**